ORDER BY

ORDER BY controls **how your result is sorted before it's shown**. It does **not** change the table—only the output.

---

1) 🔹 **Basic Syntax**

SELECT columns

FROM table

ORDER BY column_name [ASC | DESC];

- **ASC** → ascending (default)
- **DESC** → descending

---

2) 🔸 **Simple Example**

Table: employees

id	name	salary
1	Ravi	50000
2	John	70000
3	Sita	60000

➤ **Sort by salary (low → high)**

SELECT * FROM employees

ORDER BY salary;

Result:

Ravi 50000

Sita 60000

John 70000

➤ **Sort by salary (high → low)**

SELECT * FROM employees

ORDER BY salary DESC;

---

3) 🧠 **Execution Order (Important)**

SQL runs roughly like this:

FROM → WHERE → SELECT → ORDER BY → LIMIT

👉 Meaning:

- First filter rows
- Then sort them

---

4) 🔹 ORDER BY Multiple Columns (Very Important)

```
SELECT *
FROM employees
ORDER BY department_id, salary DESC;
```

How it works:

1. Sort by department_id
2. If same department → sort by salary (high → low)

Example

name	dept	salary
A	101	50000
B	101	70000
C	102	60000

Result:

Dept 101 → B (70000), A (50000)

Dept 102 → C

---

5) 🔸 ORDER BY with Expressions

```
SELECT name, salary * 12 AS yearly_salary
FROM employees
ORDER BY yearly_salary DESC;
```

👉 You can sort using calculated values.

---

6) 🔹 ORDER BY with WHERE

```
SELECT *
FROM employees
WHERE department_id = 101
ORDER BY salary DESC;
```

👉 Filter first → then sort

---

7) 🔥 ORDER BY + LIMIT (Top Results)

Top 3 highest salaries

```
SELECT *
FROM employees
ORDER BY salary DESC
LIMIT 3;
```

👉 Very common in interviews

---

8) 🔸 ORDER BY by Column Position (Not Recommended)

```
SELECT name, salary
FROM employees
ORDER BY 2;
```

👉 2 = second column (salary)

⚠️ Avoid in real projects (confusing)

---

9) 🔹 Sorting Strings

```
SELECT name
FROM employees
ORDER BY name;
```

Sorts alphabetically.

⚠️ Case sensitivity depends on **collation**.

---

10) ⚠️ NULL Handling

```
SELECT * FROM employees
```

```
ORDER BY salary;
```

- NULLs come **first** in ASC
- NULLs come **last** in DESC

---

11) 🔥 Custom Sorting (Advanced)

```
SELECT name, department_id
FROM employees
ORDER BY
  CASE
    WHEN department_id = 101 THEN 1
    WHEN department_id = 102 THEN 2
    ELSE 3
  END;
```

👉 You define your own order

---

12) 🚀 Performance Insight

- Sorting large data = expensive
- Use index if possible:

```
CREATE INDEX idx_salary ON employees(salary);
```

---

13) ❌ Common Mistakes

❌ **Wrong order**

```
SELECT * FROM employees
ORDER BY salary
WHERE department_id = 101;
```

✓ **Correct**

```
SELECT * FROM employees
WHERE department_id = 101
ORDER BY salary;
```

---

14) 🧠 Mental Model

Think like this:

1. Get data
2. Filter it
3. Choose columns
4. Sort it (ORDER BY)
5. Limit results

---

15) 🔥 Real Interview Examples

Highest salary

```
SELECT *
FROM employees
ORDER BY salary DESC
LIMIT 1;
```

Sort by name length

```
SELECT name
FROM employees
ORDER BY LENGTH(name);
```

Latest records (assuming date column)

```
SELECT *
FROM employees
ORDER BY created_at DESC;
```

🎯 One-Line Summary

ORDER BY sorts the final result of a query based on one or more columns.

💬 Straight Advice

Most people “know ORDER BY” but fail when:

- multiple columns involved

- using with LIMIT
- sorting computed values

COUNT() in MySQL

COUNT() is an **aggregate function** used to **count rows or values**.

🎯 **Basic Syntax**

SELECT COUNT(expression)

FROM table;

👉 It returns a **single number**

🔑 **Core Idea**

COUNT counts **non-NULL values**

🧩 **Types of COUNT()**

There are **3 important forms** you must understand:

1️⃣ **COUNT(*)**

SELECT COUNT(*) FROM employees;

✓ **What it does:**

- Counts **all rows**
- Includes NULLs

Example

id	name	salary
1	John	50000
2	Ravi	NULL
3	Sita	60000

Result:

3

👉 Counts all rows

2️⃣ COUNT(column_name)

SELECT COUNT(salary) FROM employees;

✓ What it does:

- Counts only **non-NULL values** in that column

Result:

2

👉 Because one salary is NULL

3️⃣ COUNT(DISTINCT column)

SELECT COUNT(DISTINCT department_id) FROM employees;

✓ What it does:

- Counts **unique values**
- Ignores duplicates

🧠 Comparison Table

Query	Counts
COUNT(*)	All rows
COUNT(column)	Non-NULL values
COUNT(DISTINCT column)	Unique non-NULL values

🔍 Important Example

Table

name	dept
A	101
B	101
C	102

name dept

D NULL

SELECT COUNT(*) FROM employees;

👉 4

SELECT COUNT(dept) FROM employees;

👉 3 (NULL ignored)

SELECT COUNT(DISTINCT dept) FROM employees;

👉 2 (101, 102)

⚠️ COUNT with Constant

SELECT COUNT('INDIA') FROM employees;

👉 Same as:

COUNT(*)

Because:

'INDIA' is never NULL

🔥 COUNT with WHERE

SELECT COUNT(*)

FROM employees

WHERE department_id = 101;

👉 Counts only matching rows

🧠 Internal Working

For:

SELECT COUNT(salary)

FROM employees;

MySQL does:

Check each row:

If salary ≠ NULL → count++

🔁 **COUNT with GROUP BY**

Count employees per department

```
SELECT department_id, COUNT(*)
FROM employees
GROUP BY department_id;
```

Example Output

dept	count
101	3
102	2

🔥 **COUNT + HAVING**

```
SELECT department_id, COUNT(*) AS total
FROM employees
GROUP BY department_id
HAVING total > 2;
```

👉 Filters grouped results

🚀 **Real Use Cases**

1. Total employees

```
SELECT COUNT(*) FROM employees;
```

---

2. Employees with salary

```
SELECT COUNT(salary) FROM employees;
```

---

3. Unique departments

```
SELECT COUNT(DISTINCT department_id) FROM employees;
```

---

4. Emails present

```
SELECT COUNT(email) FROM employees;
```

⚠️ **Common Mistakes**

❌ **Mistake 1**

COUNT(column)

Thinking it counts all rows

👉 Wrong — ignores NULL

❌ **Mistake 2**

Using COUNT for conditions incorrectly

Better:

```
SELECT COUNT(*)
FROM employees
WHERE salary > 50000;
```

💡 **Performance Insight**

- COUNT(*) is optimized in MySQL
- Faster than COUNT(column) in many cases

🧠 **Mental Model**

COUNT(*) → count rows

COUNT(column) → count non-null values

COUNT(DISTINCT) → count unique values

🔥 **Interview-Level Questions**

Q: Difference between COUNT(*) and COUNT(column)?

👉 Answer:

- COUNT(*) counts all rows
- COUNT(column) ignores NULL values

Q: What does COUNT(1) do?

👉 Same as COUNT(*)

🎯 **One-Line Summary**

COUNT() counts rows or values depending on how you use it, and it always ignores NULL unless using COUNT(*)

💬 **Straight Advice**

Master these 3:

COUNT(*)

COUNT(column)

COUNT(DISTINCT column)

👉 90% of real-world use cases covered.